

Welcome back to CS 2100!

Prof. Rasika Bhalerao

Poll: What does this print? `print("I am so excited" + "!" * 3)`

1. I am so excited! I am so excited! I am so excited!
2. I am so excited!I am so excited!I am so excited!
3. I am so excited!!!
4. Nothing – it breaks

Fun with Python: `str` times `int`

Let's practice writing tests for this function:

```
def get_cat_phrase(num: int) -> str:
    """Return a string describing the number of cats, formatted
    like: '<num> cats' (where the 's' is only added if the number
    of cats is not 1)
```

Args:

```
    num : int
        The number of cats
```

Returns:

```
    str : A string representing the number of cats
```

Raises:

```
    ValueError if the number of cats is negative
    """
```

```
    if num < 0:
        raise ValueError(f'Cannot have {num} cats')
    else:
        return f'{num} cat{"s" if num != 1}'
```

Float

Normal activity: run this:

```
print(5 / 2)
print(type(5 / 2))
```

```
2.5
<class 'float'>
```

Fun activity: run this:

```
print(4 / 2)
print(type(4 / 2))
```

Formatting floats

Let's figure out how to print `price` with exactly two decimal places.

Starting point:

```
price = 12.345678
print(f'{price}')
```

Evaluate:

`7 < 2 + 8`

Evaluate:

`3 < 4 and 5 < 7`

Order of operations

- Math happens before comparison operations
- Comparison happens before boolean operations

Mutation testing: how we autograde your tests

"Mutation testing": programmers insert small, common bugs into their code and check whether the tests catch them.

We grade student tests by checking that they:

1. Pass on correct code
2. Fail on incorrect code

(1) is needed before moving on to (2)

Disclaimer: All bugs that we inserted into the incorrect code are intended to be simple and common. If the autograder says there is a bug that remains undetected by your tests, look for large missing test cases, rather than digging into obscure ways code can run incorrectly.

```
def add(a: int, b: int) -> int:
    """Returns the sum of two integers."""
    return a + b

def test_add_positive_numbers() -> None:
    """Test adding two positive numbers."""
    assert False

def test_add_negative_numbers() -> None:
    """Test adding two negative numbers."""
    assert add(-1, -1) == -2

def test_add_mixed_numbers() -> None:
    """Test adding a positive and a
    negative number."""
    assert add(-1, 1) == 0

def test_add_zero() -> None:
    """Test adding zero to a number."""
    assert add(0, 5) == 5
    assert add(5, 0) == 5
```

Poll: Why is this assignment submission not receiving full points?

1. The student implemented `add()` incorrectly.
2. Mutation testing: the student's tests don't cover enough cases.
3. The student's test fails on correct code, so mutation tests are not run.
4. It's something else -- pylint warnings, infinite loop, etc.

How many tests do I need?

For the grade? Enough to pass the mutation tests

In life? ... Consider all the ways the function might behave:

- Normal / happy case (expected inputs)
 - `assert 5 == add(2, 3)`
 - `assert 1 == add(2, 3)`
 - `assert 'A' == calculateGrade(96)`
- Invalid inputs
 - `with pytest.raises(ValueError): calculateGrade(-600)`
 - `with pytest.raises(ValueError): add('two', 3)`
 - `with pytest.raises(ValueError): get_area_of_rectangle(-1, 4)`

Edge cases

- Edge cases at the boundaries (almost invalid, but not quite)
 - `assert 0 == get_area_of_rectangle(0, 4)`
 - `assert 0 == divide(0, 1)`

If the function has conditionals, make sure to have test cases for each branch.

Poll: We're testing a function `calculateGrade(score: int)` → `str` that returns a letter grade given a percentage. Which test case is most important to include?

1. `assert 'B+' == calculateGrade(87)`
2. `assert 'F' == calculateGrade(0)`
3. `with pytest.raises(ValueError): calculateGrade(-600)`
4. All of these are equally important



Bill Sempf

@sempf

QA Engineer walks into a bar. Orders a beer. Orders 0 beers. Orders 9999999999 beers. Orders a lizard. Orders -1 beers. Orders a sfdeljknesv.

Iteration: for loops over range of numbers

Helpful function: `range()`

```
for i in range(4):  
    print(i)
```

```
>> 0  
    1  
    2  
    3
```

Can start at a number other than 0:

```
for i in range(2, 5):  
    print(i)
```

```
>> 2  
    3  
    4
```

Can count in "steps" larger than 1:

```
for i in range(10, 30, 5):  
    print(i)
```

```
>> 10  
    15  
    20  
    25
```

What's the silliest way you can count from 1 to 10? Inspiration:

```
for i in range(-30, -130, -10):  
    print(int(-(i + 30) * 0.1) + 1)
```

Iteration: for loops over the elements of a collection

The `range()` function returns a collection, which the for loop iterates over.

Can iterate over the elements of a different collection:

```
for character in 'I love cats!':  
    print(character.upper())
```

```
I  
  
L  
O  
V  
E  
  
C  
A  
T  
S  
!
```

```
def sarcasm(phrase: str) -> str:
    """Returns the sarcastic version
    of the provided phrase, where a
    randomly selected half of the
    characters are uppercase, and
    the others are lowercase.
```

Parameters

phrase : str

The phrase to turn sarcastic

Returns

str

The sarcastic version of
the phrase

"""

```
sarcastic_phrase = ''
```

```
for character in phrase:
```

```
    if random() < 0.5:
```

```
        sarcastic_phrase +=  
            character.upper()
```

```
return sarcastic_phrase
```

Poll: What's wrong? Why doesn't the docstring match the code?

1. It's adding the index of the character, not the character itself
2. It skips adding about half of the letters
3. Sometimes, it doesn't return a string at all
4. It adds extra characters to the string

Note: Data structures (list and set)

We will have lectures dedicated to lists and sets later on, but here is the basic syntax to create them:

```
nums: list[int] = [1, 2, 3]
words: set[str] = {'hi', 'hello', 'howdy'}
```

We're assuming you have used lists before. Sets may be new to some.

A set is very similar to a list: it is a collection of items.

Differences between a set and a list:

- A set is unordered
- A set can only hold each item (at most) once -- no duplicates

What if I want the elements *and* the indices?

Use `enumerate()` :

```
for index, word in enumerate(['American Shorthair', 'Balinese', 'Cheetah']):  
    print(f'{index}: {word}')
```

```
>> 0: American Shorthair  
    1: Balinese  
    2: Cheetah
```

More things used in assignments...

Refactoring

Refactoring is moving the code around without changing the functionality

Programmers refactor their code to make it more readable, more testable, and easier to modify

We often refactor...

- Code used in multiple places into a single function that gets called multiple times
- Code from a complex function into smaller functions
- (Magic) numbers or string literals into constants

Magic numbers

Magic numbers are unnamed numeric literals in code.

We don't like magic numbers.

We name our literals (except for -1, 0, 1, and 2) to make our code self-documenting.

We name our literals by making them into **constants**: variables named in

`UPPER_SNAKE_CASE` that aren't meant to be modified while the programming is running.

Why use named constants?

- Readability

```
SECONDS_PER_MINUTE = 60  
MINUTES_PER_HOUR = 60  
HOURS_PER_DAY = 24  
SECONDS_PER_DAY = SECONDS_PER_MINUTE * MINUTES_PER_HOUR * HOURS_PER_DAY
```

- Safety

```
timer(SECONDS_PER_DAY)  
timer(86400)
```

- Maintainability

```
CREDITS_TO_GRADUATE = 128  
NUMBER_OF_CAMPUSES = 10
```

Import code

Import modules like `import pytest`

Can also import code from a file that we wrote ourselves: `import my_file`

When a Python file is imported, all of the code inside it is executed. That's why we put our code inside functions -- we don't want the code inside to be executed when it's imported!

Call all the functions in `main()` and add this at the end of the file:

```
if __name__ == '__main__':  
    main()
```

Import code: trying it out in lecture

1. Put `print('hello')` in a new file, import it in this file, and run this file (where it is imported)
2. Move it to inside a function and run it
3. Add the `if __name__ == '__main__':` conditional at the end and run it
4. Run this file (where it is imported) to make sure it doesn't print

The CS 2100 Semester Project

Project Guidelines:

https://drive.google.com/file/d/1AJXr8XDEZP_mDH9NwIMhYbHJVMzeVe28/view?usp=sharing

There are four types of assessments this semester:

- Weekly assignments (independent question sets)
- Five quizzes (in class on paper)
- In-class exercises (mostly polls)
- **The Semester Project: an open-ended project where you get to choose how to incorporate the required topics**

The CS 2100 Semester Project

Project Guidelines:

https://drive.google.com/file/d/1AJXr8XDEZP_mDH9NwIMhYbHJVMzeVe28/view?usp=sharing

Why project? It's a shift from:	To:
"Write this exact essay," but for code	"Practice using this vocabulary word in a sentence"
Entirely autograded work	Grades come from discussing your work with a TA
All students have the same project on their resume	Students get to design their projects

Poll:

- 1. What is your main takeaway from today?**
- 2. What would you like to revisit next time?**